

Step 4: Testing

BCS3210 Blockchains group project: Decentralized Digital Identity and Data Sharing Platform

What we did in this step

We wrote unit tests in Solidity (forge-std, like in Lab 3) for every contract, plus integration tests for whole workflows. We measured the gas cost of deployment and of every function, then made the contracts cheaper by reducing storage writes and measured again. All tests use made-up addresses (makeAddr("patient")) and fake hashes, so no personal data is used.

Run with `npx hardhat test` (gas table: `npx hardhat test --gas-stats`, scenario report: `npx hardhat run scripts/gas-report.ts`).

1. Test results

64 tests, 64 passing, 0 failing (one of them is a fuzz test with 256 random runs).

Test file	Tests	Passed
AccessToken.t.sol	10	10
DigitalIdentity.t.sol	16	16
ConsentManager.t.sol	21	21
DataSharing.t.sol	13	13
Integration.t.sol	4	4
Total	64	64

2. What we test and why it matters

Functionality (tests)	Why it is critical for the platform
AccessToken: only the minter can mint, not even the owner (test_RevertsWhenNonMinterMints), only the owner sets the minter, transfer / approve / transferFrom work and fail without balance or allowance	If anyone could mint, the reward would be worthless and could be abused. The token must behave like a normal ERC-20 so patients can use their reward
Registration: stores the right hashes and reference, emits an event, no double registration, empty hashes / reference rejected, a requester cannot also be a patient	Registration is the base of the identity. Wrong or empty hashes would make the integrity check of the photos useless
Requester approval: only the admin can approve or remove, cannot approve a patient or approve twice	Consent can only go to real healthcare providers, a patient cannot give access to a random address by mistake
Document update: only a registered patient, hashes change	IDs expire; patients must be able to renew without losing their account
Granting consent: stores scope and end time, event, only registered patients, only approved requesters, 0 and 366 days rejected, 1 and 365 accepted, fuzz test for all durations, granting again replaces the consent	This is the core rule of the brief: time-limited consent for a specific data type. Boundary tests catch off-by-one errors
Rewards: 10 ACT on the first consent, nothing for re-granting to the same requester, reward for each new requester, only admin changes the amount	Users must be rewarded, but it must not be possible to farm tokens by granting and revoking in a loop

Functionality (tests)	Why it is critical for the platform
Revoking: works and emits an event, cannot revoke nothing, twice or after expiry, a requester cannot revoke for the patient	Patients must be able to take access back at any time, and only they can do it
Expiry: valid one second before expiresAt, invalid at expiresAt	Expiry is lazy (no transaction), so the time comparison must be exactly right
Access granted: returns true, log entry has requester, scope, result and time, AccessGranted event	The off-chain vault only releases files after this event, so it must be correct
Access denied: no consent, revoked, expired, requester not approved or removed: returns false, does not revert, entry with the right reason, AccessDenied event	The brief requires failed attempts to be logged. A revert would erase the log entry, so this is the most important property
Audit log: all attempts kept in order, reading a wrong index reverts	The log is the proof for the patient who accessed their data
No tokens move during access	The brief says tokens must not be transferred during data access
Pause: only admin, blocks requests, unpause works	Emergency stop if something goes wrong
Integration: full flow (no consent, grant, access, revoke, re-grant, expiry) with 5 log entries; consent is per patient and per requester; removed requester loses access; document update keeps consent	Shows that the four contracts work together as in the real workflow: register, grant consent, access data, revoke consent

3. Gas measurement

We measured with `scripts/gas-report.ts`. It deploys the contracts on the Hardhat network and runs a fixed scenario with 10 patients and 2 doctors (10 or 30 calls per operation). Costs in ETH are $\text{gas} \times \text{gas price}$; we show 20 gwei as an example price.

Deployment cost (after optimisation)

Contract	Gas	ETH at 20 gwei
AccessToken	721,687	0.0144
DigitalIdentity	850,665	0.0170
ConsentManager	774,213	0.0155
DataSharing	774,276	0.0155
setMinter (wiring)	47,302	0.0009
Total	3,168,143	0.0634

Gas per function (after optimisation)

Function	Calls	Min	Avg	Max	ETH at 20 gwei (avg)
approveRequester	2	49,680	49,686	49,692	0.00099
registerUser	10	144,211	145,921	161,311	0.00292
grantConsent (first, with reward)	10	93,125	94,835	110,225	0.00190
grantConsent (again, no reward)	10	39,283	39,283	39,283	0.00079
revokeConsent	10	28,969	28,969	28,969	0.00058
requestAccess (granted)	30	70,097	70,108	70,109	0.00140

Function	Calls	Min	Avg	Max	ETH at 20 gwei (avg)
requestAccess (denied, first log entry)	10	87,279	87,290	87,291	0.00175
requestAccess (denied, revoked)	10	70,199	70,210	70,211	0.00140
requestAccess (denied, not approved)	10	63,402	63,413	63,414	0.00127
updateDocument	10	39,357	39,364	39,369	0.00079

The first registerUser and grantConsent in the whole system cost more (the Max column) because counters and the token supply are written from zero for the first time.

4. Optimisation

requestAccess is called most often, so we looked at storage writes first (a new storage slot costs 20,000 gas, changing one costs 2,900 to 5,000). Changes:

1. **Struct packing.** Timestamps are stored as uint64 instead of uint256. Now Consent (scope, grantedAt, expiresAt, revoked, exists, rewarded) and AccessLog (requester, scope, result, reason, timestamp) each fit in **one** 32-byte slot instead of 4 and 2. In User, registeredAt and registered share one slot.
2. **Reward flag inside the consent.** The separate rewarded mapping was one more new slot on the first grant; now it is a bool in the packed Consent.
3. **immutable contract references** (identity, token, consentManager). They are set once in the constructor, so reading them is free instead of a storage read.
4. **Removed the totalRequests counter.** It was one extra storage write per request and the same number can be counted from the events off-chain.

Operation	Before	After	Change
requestAccess (granted)	107,429	70,108	-34.7%
requestAccess (denied, revoked)	107,531	70,210	-34.7%
requestAccess (denied, not approved)	92,550	63,413	-31.5%
requestAccess (denied, first log entry)	126,321	87,290	-30.9%
grantConsent (first, with reward)	187,290	94,835	-49.4%
grantConsent (again, no reward)	53,229	39,283	-26.2%
registerUser	167,975	145,921	-13.1%
revokeConsent	30,938	28,969	-6.4%
Deployment, all contracts	3,140,723	3,168,143	+0.9%

Deployment became slightly more expensive (extra casts and the rewarded getter), but deployment happens once while access requests happen all the time, so this trade-off is worth it. A uint64 holds timestamps for billions of years, so nothing is lost by the smaller type.

All 64 tests still pass after the optimisation.